

Unit-5 DeadLock

Deadlock Concept

1. Introduction: Deadlock is one of the most important and dangerous problems in Operating Systems. In a multiprogramming system, multiple processes compete for limited resources such as:

- CPU
- Memory
- Printer
- Files
- I/O devices

Sometimes processes wait for resources held by each other, and none of them can proceed further. This situation is called Deadlock.

Definition: A Deadlock is a situation in which two or more processes wait indefinitely for resources held by each other.

Simple Exam Definition: Deadlock is a condition where processes are permanently blocked because each process is waiting for a resource held by another process.

Basic Idea of Deadlock

Suppose:

- Process P1 holds Resource R1 and waits for R2
- Process P2 holds Resource R2 and waits for R1

Now:

- P1 cannot proceed
- P2 cannot proceed

Both wait forever.

This creates a deadlock.

Simple Deadlock Diagram

P1 → waiting for R2

P2 → waiting for R1

Real-Life Analogy

Traffic Junction Example

Imagine:

- Four cars enter a four-way junction
- Each car waits for another car to move

No car can move.

This is similar to deadlock.

Another Real-Life Example

Pen and Paper Example

Suppose:

- Student A has pen and needs paper
- Student B has paper and needs pen

Both wait forever.

Deadlock occurs.

Why Deadlock Happens

Deadlock occurs because:

- resources are limited
- processes compete for resources
- improper resource allocation occurs

Resources in Deadlock

A resource is anything required by a process for execution.

Types of Resources

Resource Type	Example
---------------	---------

Hardware Resources	CPU, printer, disk
Software Resources	Files, semaphores
Data Resources	Database records

Process-Resource Interaction

A process uses resources in this order:

Request → Use → Release

Example

Process requests printer



Uses printer



Releases printer

Deadlock Situation

Deadlock occurs when:

Processes hold some resources while waiting for others.

Conditions for Deadlock (Overview)

Four conditions are required for deadlock.

Condition	Meaning
Mutual Exclusion	Resource used by one process only
Hold and Wait	Process holds and waits
No Preemption	Resource cannot be forcibly removed
Circular Wait	Processes form waiting cycle

These topics are studied in detail later.

Characteristics of Deadlock

Characteristic	Explanation
Permanent waiting	Processes never finish
Resource holding	Processes keep resources
Circular dependency	Processes depend on each other
No progress	System work stops

Example of Deadlock

Suppose:

Resources

- Printer
- Scanner

Processes

- P1
- P2

Situation

Process P1

- Holds Printer
- Requests Scanner

Process P2

- Holds Scanner
- Requests Printer

Diagram

P1 → Printer → waiting for Scanner

P2 → Scanner → waiting for Printer

Neither process can continue.

Deadlock occurs.

Deadlock in Operating Systems

Deadlocks commonly occur in:

- multiprogramming systems
- databases
- distributed systems
- file systems

Types of Deadlocks

Type	Description
Resource Deadlock	Processes wait for resources
Communication Deadlock	Processes wait for messages
Distributed Deadlock	Deadlock across multiple systems

Resource Deadlock

Most common type.

Occurs due to:

- printers
- memory
- files
- devices

Communication Deadlock

Occurs when:

- processes wait for messages from each other

Example:

P1 waits for message from P2

P2 waits for message from P1

Distributed Deadlock

Occurs in distributed systems where processes on different computers wait for resources.

Effects of Deadlock

Effect	Explanation
System slowdown	Processes stop progressing
Resource wastage	Resources remain occupied
Reduced throughput	Less work completed
Process starvation	Other processes may suffer
System crash possible	Severe deadlocks may freeze system

Deadlock vs Starvation

Feature	Deadlock	Starvation
Waiting	Forever	Very long
Cause	Circular waiting	Unfair allocation
Progress	No process progresses	Some processes progress

Deadlock vs Livelock

Feature	Deadlock	Livelock
Process state	Blocked	Active but useless
Resource usage	Waiting	Continuously changing state
Progress	None	None

Deadlock Handling Approaches

Operating systems use four methods.

Method	Purpose
Prevention	Prevent deadlock conditions
Avoidance	Avoid unsafe state
Detection	Detect deadlock occurrence

Recovery	Remove deadlock
-----------------	-----------------

Deadlock Handling Flow

Deadlock



Prevention

Avoidance

Detection

Recovery

Important Terms

Term	Meaning
Process	Program in execution
Resource	Required execution object
Allocation	Resource assigned
Request	Resource demanded
Waiting	Process blocked

Important Exam Points

- Deadlock means permanent waiting
- Occurs due to improper resource allocation
- Processes wait for each other's resources
- Four conditions are necessary
- Deadlock affects system performance
- OS uses prevention, avoidance, detection, and recovery methods

Advantages of Studying Deadlocks

Advantage	Explanation
Better resource management	Efficient allocation
Safer system design	Avoid system freeze
Improved synchronization	Better process coordination
OS reliability	Stable execution

Disadvantages of Deadlock

Disadvantage	Explanation
System freeze	Processes blocked
Reduced efficiency	No useful work
Resource wastage	Locked resources
Difficult debugging	Hard to identify

Key Terms

Term	Meaning
Deadlock	Permanent waiting
Resource	Required object
Circular Wait	Processes waiting cyclically
Allocation	Resource assignment
Blocking	Waiting state

Short Conclusion

Deadlock is a serious problem in operating systems where multiple processes wait indefinitely for resources held by each other. It occurs mainly because of improper resource allocation and circular waiting among processes. Deadlocks reduce system performance and may completely stop process execution. Operating systems use prevention, avoidance, detection, and recovery techniques to handle deadlocks efficiently.

System Model in Deadlock

1. Introduction

To understand deadlock properly, we must first understand how processes and resources interact inside an operating system.

The System Model describes:

- processes
- resource types
- resource allocation
- process-resource interaction

This model helps explain how deadlocks occur.

Definition

A System Model is a representation of processes, resources, and their interactions in a computer system.

Simple Exam Definition

The system model describes how processes request, use, and release resources in an operating system.

Basic Components of System Model

The deadlock system model mainly contains:

Component	Description
Processes	Programs in execution
Resources	Required system objects
Resource Instances	Number of available units
Resource Allocation	Assignment of resources

1. Processes

A process is a program currently executing.

Processes need resources to perform tasks.

Example:

- printing
- file access
- memory usage

2. Resources

Resources are objects required by processes for execution.

Types of Resources

Resource Type	Examples
CPU Resources	Processor time
Memory Resources	RAM
I/O Devices	Printer, Scanner
Software Resources	Files, Semaphores

Resource Instances

A resource may have:

- single instance
- multiple instances

Example

Single Instance Resource

One Printer

Only one process can use it at a time.

Multiple Instance Resource

5 Memory Blocks

Many processes may use different blocks simultaneously.

Process-Resource Interaction

A process interacts with resources in three steps:

Request → Use → Release

Step 1: Request

Process asks for resource.

Example:

P1 requests Printer

Step 2: Use

OS allocates resource.

Process performs task.

Example:

P1 prints document

Step 3: Release

Process releases resource after completion.

Example:

P1 releases Printer

Resource Allocation

The operating system manages:

- allocation
- deallocation
- scheduling of resources

Resource Allocation Example

Process Allocated Resource

P1 Printer

P2 Scanner

Deadlock Situation in System Model

Deadlock occurs when:

Processes hold resources
while waiting for others.

Example of Deadlock

Initial Allocation

Process Holding Waiting For

P1 Printer Scanner

P2 Scanner Printer

Neither process can proceed.

Deadlock occurs.

Resource Allocation State

The state of the system is determined by:

- allocated resources
- available resources
- pending requests

Types of Resource Usage

Usage Type	Meaning
------------	---------

Shared Resource	Multiple processes allowed
-----------------	----------------------------

Non-Shared Resource	One process only
---------------------	------------------

Shared Resources

Example:

Read-only file

Multiple processes can access simultaneously.

Non-Shared Resources

Example:

Printer

Only one process uses at a time.

Reusable Resources

Reusable resources can be reused after release.

Examples:

- CPU
- printer
- memory

Consumable Resources

Destroyed after use.

Examples:

- messages
- signals
- interrupts

System Model Diagram

Processes $\leftrightarrow$ Resources

|
Request / Allocate / Release

Operating System Role

The OS is responsible for:

- managing resources
- allocating resources
- preventing conflicts
- handling deadlocks

Characteristics of System Model

Characteristic	Explanation
Multiple processes	Concurrent execution
Resource sharing	Common resource usage
Dynamic allocation	Resources change continuously
Waiting possible	Processes may block

Importance of System Model

Importance	Explanation
Understand deadlocks	Core foundation
Resource management	Efficient allocation
Process coordination	Synchronization
Deadlock analysis	Detection and prevention

Advantages of Proper Resource Management

Advantage	Explanation
Better CPU utilization	Efficient execution
Improved throughput	More work completed
Reduced waiting	Faster execution
Stable system	Fewer deadlocks

Problems in System Model

Improper resource allocation may cause:

- deadlock
- starvation
- resource wastage
- blocking

Key Terms

Term	Meaning
------	---------

Process	Executing program
---------	-------------------

Resource	Required object
----------	-----------------

Allocation	Resource assignment
------------	---------------------

Request	Resource demand
---------	-----------------

Release	Returning resource
---------	--------------------

Important Exam Points

- System model explains process-resource interaction
- Resources may have single or multiple instances
- Process flow:
Request → Use → Release
- Improper allocation causes deadlock
- OS manages resource allocation

Short Conclusion

The System Model in deadlock describes how processes interact with resources in an operating system. Processes request, use, and release resources during execution. Deadlocks occur when processes hold resources while waiting for others. Understanding the system model is essential for studying deadlock prevention, avoidance, detection, and recovery techniques.

Necessary Conditions for Deadlock

1. Introduction: Deadlock does not occur randomly.

Certain conditions must exist simultaneously for deadlock to happen.

These are called:

Necessary Conditions for Deadlock

All four conditions must be present at the same time for a deadlock to occur.

If even one condition is absent:

Deadlock cannot occur

Definition: Necessary conditions for deadlock are the conditions that must exist simultaneously in a system for deadlock to occur.

Simple Exam Definition : Deadlock occurs only if four necessary conditions exist simultaneously.

Four Necessary Conditions

No.	Condition
1	Mutual Exclusion
2	Hold and Wait
3	No Preemption
4	Circular Wait

Memory Trick

Remember:

MHNC

- M → Mutual Exclusion
- H → Hold and Wait
- N → No Preemption
- C → Circular Wait

1. Mutual Exclusion

Definition: At least one resource must be non-shareable.
Only one process can use the resource at a time.

Simple Meaning

If one process is using a resource:

- other processes must wait

Example

Printer

Only one process can print at a time.

Diagram

P1 using Printer

P2 waiting

Why It Causes Deadlock

If processes cannot share resources:

- waiting begins
- deadlock possibility increases

Shared vs Non-Shared Resource

Resource Type	Deadlock Possibility
Shared Resource	Low
Non-Shared Resource	High

2. Hold and Wait

Definition: A process holds at least one resource while waiting for additional resources.

Simple Meaning: Process already has something but still requests more.

Example

P1 holds Printer

P1 waits for Scanner

Diagram

P1 → Printer

|

waiting for Scanner

Why It Causes Deadlock

Processes keep resources occupied while waiting for others.

This blocks other processes.

3. No Preemption

Definition: Resources cannot be forcibly removed from a process.

The process must release resource voluntarily.

Simple Meaning

OS cannot take resource back automatically.

Example

If:

P1 using Printer

OS cannot suddenly remove printer from P1.

Why It Causes Deadlock

Processes continue holding resources for long time.

Other processes remain blocked.

4. Circular Wait

Definition: A circular chain of processes exists where each process waits for a resource held by the next process.

Simple Meaning: Processes wait in a cycle.

Example

P1 waits for P2

P2 waits for P3

P3 waits for P1

Diagram

P1 → P2 → P3 → P1

Why It Causes Deadlock

No process can proceed because all are waiting.

Combined Deadlock Example

Process	Holding	Waiting For
P1	Printer	Scanner
P2	Scanner	Printer

Condition Analysis

Condition	Exists?
Mutual Exclusion	Yes
Hold and Wait	Yes
No Preemption	Yes
Circular Wait	Yes

Since all four conditions exist:
Deadlock occurs

Relationship Among Conditions

All four conditions are dependent.

All conditions together → Deadlock

Removing any one condition prevents deadlock.

Breaking Conditions to Prevent Deadlock

Condition Removed	Result
Mutual Exclusion removed	Shared resources
Hold and Wait removed	No partial holding
No Preemption removed	Force resource release
Circular Wait removed	Ordered resource allocation

Real-Life Analogy

Dining Table Example

Suppose:

- Four people each hold one spoon
- Everyone waits for another spoon

Conditions:

- spoon cannot be shared
- each person holds one spoon
- spoons cannot be taken away
- circular waiting exists

Deadlock occurs.

Characteristics of Necessary Conditions

Characteristic	Explanation
Simultaneous existence	All required
Resource dependency	Processes depend on resources
Waiting state	Processes blocked

Circular dependency	Waiting chain exists
----------------------------	----------------------

Importance of Necessary Conditions

Importance	Explanation
Understand deadlock	Core foundation
Prevention techniques	Break conditions
System design	Better resource management
OS stability	Avoid system freeze

Necessary Conditions vs Deadlock Prevention

Condition	Prevention Idea
Mutual Exclusion	Share resources
Hold and Wait	Request all resources together
No Preemption	Force release
Circular Wait	Resource ordering

Important Exam Points

- All four conditions are necessary
- Deadlock occurs only when all conditions exist
- Removing one condition prevents deadlock
- Circular wait creates process cycle
- No preemption means forced removal not allowed

Key Terms

Term	Meaning
Mutual Exclusion	One process at a time
Hold and Wait	Holding while requesting
No Preemption	Cannot forcibly remove
Circular Wait	Cyclic waiting

Short Conclusion

Deadlock occurs only when four necessary conditions exist simultaneously: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. These conditions create a situation where processes wait indefinitely for resources held by each other. Understanding these conditions is essential because deadlock prevention techniques are based on breaking one or more of these conditions.

Resource Allocation Graph (RAG)

1. Introduction

A Resource Allocation Graph (RAG) is a graphical method used to represent the relationship between:

- processes
- resources
- resource allocation
- resource requests

It helps visualize deadlocks in a system.

Definition

A Resource Allocation Graph is a directed graph used to represent resource allocation and process requests in an operating system.

Simple Exam Definition

A Resource Allocation Graph is a graphical representation of processes and resources used for deadlock analysis.

Purpose of Resource Allocation Graph

RAG helps:

- understand process-resource interaction
- detect deadlocks
- analyze waiting conditions
- visualize resource allocation

Components of Resource Allocation Graph

A graph contains:

Component	Representation
Process	Circle
Resource	Rectangle
Allocation Edge	Arrow from resource to process
Request Edge	Arrow from process to resource

Process Representation

Processes are represented using circles.

Example:

(P1)

(P2)

Resource Representation

Resources are represented using rectangles.

Example:

[R1]

[R2]

Edges in RAG

There are two important edges.

Edge Type	Meaning
Request Edge	Process requests resource
Allocation Edge	Resource allocated to process

1. Request Edge

Arrow direction:

Process → Resource

Meaning:

- process is waiting for resource

Example

P1 → R1

P1 requests R1.

2. Allocation Edge

Arrow direction:

Resource → Process

Meaning:

- resource assigned to process

Example

R1 → P1

R1 allocated to P1.

Example of Resource Allocation Graph

Suppose:

Process Status

P1 Holds R1 and requests R2

P2 Holds R2 and requests R1

Graph Representation

R1 → P1 → R2

R2 → P2 → R1

Circular Structure

$P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$

This forms a cycle.

Cycle in Resource Allocation Graph

A cycle is very important in deadlock analysis.

Rule for Single Instance Resources

If:

Cycle exists

Then:

Deadlock exists

Rule for Multiple Instance Resources

If cycle exists:

- deadlock may or may not exist

Additional analysis required.

Types of Resource Allocation Graph

Type	Description
Single Instance Graph	One instance per resource
Multiple Instance Graph	Multiple resource units

Single Instance Example

One Printer

One Scanner

Cycle directly indicates deadlock.

Multiple Instance Example

3 Printers

5 Memory Blocks

Cycle may not always indicate deadlock.

Resource Allocation Graph with Single Instance Example

Resources

- R1
- R2

Processes

- P1
- P2

Situation

Process Holding Waiting For

P1	R1	R2
P2	R2	R1

RAG Diagram

$R1 \rightarrow P1 \rightarrow R2$

$R2 \rightarrow P2 \rightarrow R1$

Analysis

A cycle exists:

$P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$

Therefore:

Deadlock exists

Resource Allocation Table

Process Allocated Requested

P1	R1	R2
P2	R2	R1

Advantages of RAG

Advantage	Explanation
Easy visualization	Understand resource allocation
Deadlock detection	Identify cycles
Process analysis	Track waiting
Educational simplicity	Easy understanding

Disadvantages of RAG

Disadvantage	Explanation
Complex for large systems	Large graphs difficult
Multiple instance difficulty	Cycle not always deadlock
Dynamic changes	Graph changes frequently

Resource Allocation Graph vs Wait-For Graph

Feature	Resource Allocation Graph	Wait-For Graph
Includes resources	Yes	No
Includes processes	Yes	Yes
Used for	General deadlock analysis	Simplified detection

Real-Life Analogy

Traffic System

Item	OS Equivalent
Cars	Processes
Roads	Resources
Waiting junction	Resource request

If cars form circular waiting:

- traffic deadlock occurs

Characteristics of RAG

Characteristic	Explanation
Directed graph	Arrows show relationship
Dynamic structure	Changes continuously
Deadlock analysis tool	Helps detection
Resource-process interaction	Visual representation

Important Exam Points

- Process = Circle
- Resource = Rectangle
- Request edge:
Process → Resource
- Allocation edge:
Resource → Process
- Cycle indicates deadlock for single-instance resources

Key Terms

Term	Meaning
Request Edge	Waiting request
Allocation Edge	Assigned resource
Cycle	Circular dependency
Resource Node	Resource representation
Process Node	Process representation

Short Conclusion

A Resource Allocation Graph is a graphical representation used in operating systems to model the allocation and request of resources by processes. It helps analyze process-resource interactions and detect deadlocks. In systems with single-instance resources, the presence of a cycle in the graph indicates deadlock.

Deadlock Prevention

1. Introduction: Deadlock Prevention is a technique used to ensure that deadlocks never occur in a system.

The basic idea is:

Break at least one necessary condition for deadlock.

Since deadlock requires all four conditions simultaneously, removing one condition prevents deadlock completely.

Definition: Deadlock Prevention is a method of handling deadlocks by ensuring that at least one necessary condition for deadlock cannot occur.

Simple Exam Definition: Deadlock prevention prevents deadlock by breaking one or more necessary conditions.

Main Idea of Prevention

Necessary conditions:

- Mutual Exclusion
- Hold and Wait
- No Preemption
- Circular Wait

If one condition is removed:

Deadlock cannot occur

Deadlock Prevention Flow

Break one condition



Deadlock impossible

Prevention Methods

Condition Broken	Prevention Technique
Mutual Exclusion	Share resources
Hold and Wait	Request all resources together
No Preemption	Force release resources
Circular Wait	Resource ordering

1. Preventing Mutual Exclusion

Idea: Make resources shareable whenever possible.

Explanation

If multiple processes can use a resource simultaneously:

- waiting reduces
- deadlock possibility decreases

Example

Shared resource:

Read-only file

Many processes can access together.

Limitation: Some resources cannot be shared.

Example:

Printer

Only one process can use it at a time.

Advantages

Advantage	Explanation
Reduces waiting	Shared access
Better resource utilization	Multiple usage

Disadvantages

Disadvantage	Explanation
Not possible for all resources	Some require exclusive access

2. Preventing Hold and Wait

Idea: A process must request all required resources at once before execution begins.

OR

A process releases held resources before requesting new ones.

Explanation

Processes never hold resources while waiting for others.

Example

Instead of:

Request Printer first

Then Scanner

Process requests:

Printer + Scanner together

Advantages

Advantage	Explanation
Deadlock prevention	No partial holding
Simpler management	Reduced circular dependency

Disadvantages

Disadvantage	Explanation
Low resource utilization	Resources remain unused
Long waiting time	Process waits for all resources

3. Preventing No Preemption

Idea

If process requests unavailable resource:

- OS forcibly removes held resources

Explanation : Resources can be preempted and reassigned.

Example

Suppose:

- P1 holds printer
- requests scanner
- scanner unavailable

OS:

takes printer back from P1

Advantages

Advantage	Explanation
Reduces waiting	Resources reused

Prevents circular dependency	Dynamic allocation
------------------------------	--------------------

Disadvantages

Disadvantage	Explanation
Difficult implementation	Some resources non-preemptable
Possible data inconsistency	Forced interruption

4. Preventing Circular Wait

Idea: Assign numerical ordering to resources.

Processes request resources only in increasing order.

Example

Resource	Order
Printer	1
Scanner	2
Disk	3

Rule

If process holds:

Printer (1)

It can request:

Scanner (2)

But not:

lower ordered resource

Why It Works

Circular chain becomes impossible.

Example

Without ordering:

$P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow P1$

With ordering:

- cyclic requests prevented

Advantages

Advantage	Explanation
Effective prevention	Removes cycles
Simple concept	Easy implementation

Disadvantages

Disadvantage	Explanation
Reduced flexibility	Fixed ordering required
Difficult resource ordering	Large systems complex

Comparison of Prevention Techniques

Technique	Main Idea
Mutual Exclusion Prevention	Share resources
Hold and Wait Prevention	Request all together
No Preemption Prevention	Force release
Circular Wait Prevention	Resource ordering

Deadlock Prevention Example

Suppose:

- Printer order = 1
- Scanner order = 2

If process already holds Scanner:

Cannot request Printer

This breaks circular wait.

Advantages of Deadlock Prevention

Advantage	Explanation
Deadlock impossible	System safety
Stable execution	No permanent waiting
Predictable behavior	Controlled allocation

Disadvantages of Deadlock Prevention

Disadvantage	Explanation
Poor resource utilization	Resources reserved unnecessarily
Reduced concurrency	Processes wait more
Lower throughput	Less parallel execution

Prevention vs Avoidance

Feature	Prevention	Avoidance
Approach	Break conditions	Avoid unsafe state
Flexibility	Lower	Higher
Resource utilization	Lower	Better

Characteristics of Deadlock Prevention

Characteristic	Explanation
Proactive approach	Stops deadlock before occurrence
Restrictive policies	Limits resource usage
Guaranteed safety	Deadlock impossible

Important Exam Points

- Prevention breaks necessary conditions
- Deadlock impossible if one condition absent
- Circular wait prevention uses resource ordering
- Hold and wait prevention requests all resources together
- Prevention reduces resource utilization

Key Terms

Term	Meaning
Prevention	Stop deadlock before occurrence
Preemption	Forced resource removal
Circular Wait	Cyclic dependency
Resource Ordering	Fixed allocation order

Short Conclusion

Deadlock Prevention is a strategy used to ensure that deadlocks never occur by breaking one or more necessary conditions required for deadlock. Techniques include resource sharing, requesting all resources together, resource preemption, and resource ordering. Although prevention guarantees system safety, it may reduce resource utilization and system efficiency.

Deadlock Avoidance

1. Introduction: Deadlock Avoidance is a technique used in operating systems to avoid deadlocks dynamically during resource allocation.

Unlike deadlock prevention:

- prevention breaks conditions permanently
- avoidance carefully allocates resources so the system never enters an unsafe state

The operating system checks every resource request before allocation.

Definition: Deadlock Avoidance is a method of handling deadlocks in which the operating system allocates resources only if the system remains in a safe state.

Simple Exam Definition : Deadlock avoidance prevents deadlock by ensuring the system never enters an unsafe state.

Main Idea of Deadlock Avoidance

Before allocating resources:

- OS analyzes current system state
- checks future possibility of deadlock
- allocates resource only if system remains safe

Deadlock Avoidance Flow

Process requests resource



OS checks system state



Safe State ?



Yes



No

Allocate Deny Request

Important Concepts

Concept	Meaning
Safe State	Deadlock-free state
Unsafe State	May lead to deadlock
Safe Sequence	Order of process completion

Safe State

Definition: A system is in a safe state if there exists at least one sequence in which all processes can complete successfully.

Simple Meaning

Processes can finish one by one without deadlock.

Example

Suppose:

- enough resources available
- processes can complete in some order

Then:

System is safe

Safe Sequence

A safe sequence is an order in which processes can execute and release resources safely.

Example

P1 → P3 → P2

All processes complete successfully.

Unsafe State

Definition: A system is in an unsafe state if no safe sequence exists.

Important Point

Unsafe state does NOT always mean deadlock.

But: Unsafe state can lead to deadlock

Relationship Between States

Safe State → Unsafe State → Deadlock

Safe vs Unsafe State

Feature	Safe State	Unsafe State
Safe sequence exists	Yes	No
Deadlock possibility	No	Possible
System stability	Stable	Risky

Example of Safe State

Suppose:

- Process P1 can finish first
- releases resources
- P2 finishes next

Safe sequence exists.

Therefore:

System is safe

Example of Unsafe State

Suppose:

- no process can complete with available resources

Then:

Unsafe state occurs

Need for Deadlock Avoidance

Deadlock avoidance:

- improves resource utilization
- increases concurrency
- avoids permanent blocking

Requirements for Deadlock Avoidance

OS must know in advance:

Requirement	Meaning
Maximum resource need	Future demand
Current allocation	Allocated resources
Available resources	Free resources

Methods of Deadlock Avoidance

Method	Purpose
Resource Allocation Graph Algorithm	Single-instance resources
Banker's Algorithm	Multiple-instance resources

Resource Allocation Graph Method

Used when: Each resource has single instance

OS avoids cycles in graph.

Banker's Algorithm

Used when:

Resources have multiple instances

Most important avoidance algorithm.

Characteristics of Deadlock Avoidance

Characteristic	Explanation
Dynamic checking	Resource requests analyzed
Future consideration	Predicts safe execution
Safe state maintenance	Avoids unsafe allocation
Better utilization	Efficient resource use

Advantages of Deadlock Avoidance

Advantage	Explanation
Prevents deadlock	Safe allocation
Better resource utilization	More efficient
Higher concurrency	More parallel execution
Flexible resource management	Dynamic allocation

Disadvantages of Deadlock Avoidance

Disadvantage	Explanation
--------------	-------------

Complex implementation	Continuous checking
Requires future information	Maximum need required
Computational overhead	Frequent analysis
Slower execution	Extra checking time

Avoidance vs Prevention

Feature	Avoidance	Prevention
Strategy	Avoid unsafe states	Break conditions
Resource utilization	Better	Lower
Flexibility	High	Low
Complexity	Higher	Lower

Real-Life Analogy

Bank Loan Example

A bank gives loans only if:

- it can safely satisfy all customers later

Otherwise:

- loan request rejected

This idea is similar to deadlock avoidance.

Important Exam Points

- Avoidance maintains safe state
- Unsafe state may lead to deadlock
- Safe sequence is essential
- Banker's Algorithm used for multiple instances
- Avoidance checks requests dynamically

Key Terms

Term	Meaning
Safe State	Deadlock-free state
Unsafe State	Risk of deadlock
Safe Sequence	Safe process execution order
Avoidance	Dynamic deadlock handling

Short Conclusion

Deadlock Avoidance is a dynamic deadlock handling technique in which the operating system allocates resources carefully to ensure the system always remains in a safe state. It avoids unsafe states by checking resource allocation requests before granting them. Banker's Algorithm is the most common deadlock avoidance algorithm used for systems with multiple resource instances.

Banker's Algorithm

1. Introduction

Banker's Algorithm is the most important deadlock avoidance algorithm in operating systems.

It was developed by:

Edsger Dijkstra

The algorithm works like a banking system:

- bank grants loans only if safe repayment is possible

Similarly:

- OS allocates resources only if system remains safe.

Definition: Banker's Algorithm is a deadlock avoidance algorithm that allocates resources only if the resulting system state remains safe.

Simple Exam Definition: Banker's Algorithm checks for safe state before allocating resources to avoid deadlock.

Main Idea

Before granting request:

- OS simulates allocation

- checks safe sequence
- grants request only if system remains safe

Why Called Banker's Algorithm?

Like a banker:

- loans are given carefully
- enough money must remain for others

OS behaves similarly with resources.

Basic Concepts

Concept	Meaning
Available	Free resources
Max	Maximum demand
Allocation	Currently allocated resources
Need	Remaining resource requirement

Important Formula

The most important formula:

Need = Max - Allocation

Data Structures Used

1. Available Matrix

Shows currently available resources.

Example:

Resource	Available
A	3
B	2

2. Max Matrix

Maximum resources each process may need.

3. Allocation Matrix

Resources already allocated to processes.

4. Need Matrix

Remaining resources required.

Calculated using:

$$Need_{ij} = Max_{ij} - Allocation_{ij}$$

Banker's Algorithm Components

Algorithm	Purpose
Safety Algorithm	Checks safe state
Resource Request Algorithm	Handles new requests

1. Safety Algorithm

Determines whether:

System is safe or unsafe

Steps of Safety Algorithm

Step 1

Find: Need = Max - Allocation

Step 2

Initialize:

Work = Available

Finish[i] = false

Step 3

Find process satisfying:

$$Need_i \leq Work$$

Step 4

If process found:

- allocate resources logically
- process completes
- release allocated resources

Update:

$$Work = Work + Allocation_i$$

Step 5

Repeat until:

- all processes finish
OR
- no suitable process found

Safe Sequence

If all processes complete:

Safe sequence exists

Example:

P1 → P3 → P0 → P2

Unsafe State

If no process satisfies condition:

Unsafe state occurs

Resource Request Algorithm

Used when process requests additional resources.

Steps**Step 1**

Check:

$$Request_i \leq Need_i$$

Step 2

Check:

$$Request_i \leq Available$$

Step 3

Temporarily allocate resources.

Step 4

Run safety algorithm.

Step 5

If safe: Grant request

Else: Rollback allocation

Banker's Algorithm Example

Suppose:

Process	Max	Allocation
P1	7	3
P2	5	2

Need:

Process	Need
P1	4
P2	3

Banker's Algorithm Diagram

Request



Safety Check



Safe ?



Yes No

Grant Reject

Advantages of Banker's Algorithm

Advantage	Explanation
Prevents deadlock	Safe allocation
Better utilization	Efficient management
Dynamic allocation	Flexible handling
Safe execution	Avoids unsafe states

Disadvantages of Banker's Algorithm

Disadvantage	Explanation
Complex calculations	Matrix operations
Requires future knowledge	Max demand needed
High overhead	Frequent checking
Slower performance	Computational cost

Banker's Algorithm vs Prevention

Feature	Banker's Algorithm	Prevention
Method	Avoid unsafe states	Break conditions
Resource utilization	Better	Lower
Complexity	Higher	Lower

Real-Life Analogy

Banking System

Bank grants loan only if:

- enough money remains
- all customers can eventually repay

Otherwise:

- loan denied

Characteristics of Banker's Algorithm

Characteristic	Explanation
Dynamic checking	Every request analyzed
Safe-state based	Prevent unsafe allocation
Multiple resource support	Handles many instances
Matrix-based algorithm	Uses tables and calculations

Important Exam Points

- Banker's Algorithm is deadlock avoidance technique
- Developed by Dijkstra
- Uses:
 - Available
 - Max
 - Allocation
 - Need
- Formula:
 $\text{Need} = \text{Max} - \text{Allocation}$
- Safe sequence required

Key Terms

Term	Meaning
Available	Free resources
Allocation	Assigned resources
Max	Maximum requirement
Need	Remaining requirement
Safe Sequence	Safe completion order

Short Conclusion

Banker's Algorithm is a deadlock avoidance technique that allocates resources carefully to ensure the system always remains in a safe state. It uses matrices such as Available, Max, Allocation, and Need to analyze resource requests dynamically. By granting requests only when a safe sequence exists, Banker's Algorithm prevents deadlocks while improving resource utilization.

Deadlock Detection

1. Introduction

Sometimes operating systems do not prevent or avoid deadlocks.

Instead:

- the system allows deadlocks to occur
- then detects them later

This approach is called:

Deadlock Detection

Definition: Deadlock Detection is a technique in which the operating system checks whether a deadlock has occurred in the system.

Simple Exam Definition: Deadlock detection identifies deadlocks after they occur.

Main Idea

The system:

- allocates resources freely
- periodically checks for deadlocks
- takes corrective action if deadlock found

Detection Flow

Resource Allocation



Deadlock Check



Deadlock Found ?



Yes

Recover



No

Continue

Why Detection is Needed

Deadlock prevention and avoidance:

- reduce performance
- reduce resource utilization

Detection allows:

- better flexibility
- higher resource usage

Conditions for Deadlock Detection

Detection checks:

- process waiting
- resource allocation
- circular dependency

Detection Methods

Method	Used For
Wait-For Graph	Single-instance resources
Detection Algorithm	Multiple-instance resources

1. Wait-For Graph Method

Used when:

Each resource has single instance

Wait-For Graph

A simplified graph containing only:

- processes
- waiting relationships

Resources are removed from graph.

Representation

$P1 \rightarrow P2$

Meaning:

P1 waits for P2

Deadlock Rule

If: Cycle exists in wait-for graph

Then: Deadlock exists

Example

$P1 \rightarrow P2$

$P2 \rightarrow P3$

$P3 \rightarrow P1$

Cycle exists.

Therefore:

Deadlock detected

Wait-For Graph Diagram

$P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$

2. Detection Algorithm

Used for: Multiple-instance resources

Data Structures Used

Structure	Purpose
Available	Free resources
Allocation	Allocated resources
Request	Current requests

Detection Algorithm Steps

Step 1

Initialize:

$Work = Available$

Step 2

For each process:

$Finish[i] = false$

Step 3

Find process satisfying:

$$Request_i \leq Work$$

Step 4

If found:

- process completes
- resources released

Update:

Work = Work + Allocation_i

Step 5

Repeat until:

- all processes finish
OR
- no process can proceed

Deadlock Detection Rule

If some processes remain unfinished:

Deadlock exists

Example

Suppose:

- P1 waits for unavailable resources
- P2 waits for P1
- no process can continue

Then:

Deadlock detected

Detection Timing

Operating system may run detection:

- periodically
- when CPU utilization decreases
- when many processes blocked

Frequency of Detection

Frequency	Effect
Frequent detection	Faster recovery
Less frequent detection	Lower overhead

Advantages of Deadlock Detection

Advantage	Explanation
Better resource utilization	Resources allocated freely
High concurrency	Less restriction
Flexible execution	Dynamic allocation

Disadvantages of Deadlock Detection

Disadvantage	Explanation
Deadlock actually occurs	Detection happens later
Recovery overhead	Extra correction needed
Complex implementation	Detection algorithms costly

Detection vs Avoidance

Feature	Detection	Avoidance
Deadlock occurrence	Allowed	Prevented
Checking time	After occurrence	Before allocation
Flexibility	Higher	Lower

Characteristics of Deadlock Detection

Characteristic	Explanation
Reactive approach	Acts after deadlock
Periodic analysis	Continuous monitoring
Graph/algorithm based	Uses system state

Recovery required

Deadlock removal needed

Real-Life Analogy

Traffic Police Example

Traffic allowed normally.

When traffic jam occurs:

- police detect problem
- clear blocked vehicles

Similar to deadlock detection and recovery.

Important Exam Points

- Detection allows deadlock occurrence
- Wait-for graph used for single-instance resources
- Cycle indicates deadlock
- Detection algorithm used for multiple instances
- Unfinished processes indicate deadlock

Key Terms

Term	Meaning
Wait-For Graph	Process dependency graph
Detection Algorithm	Finds deadlocks
Work	Available working resources
Finish	Process completion status

Short Conclusion

Deadlock Detection is a deadlock handling technique in which the operating system allows deadlocks to occur and later identifies them using graphs or detection algorithms. Wait-for graphs are used for single-instance resources, while detection algorithms are used for multiple-instance resources. Once detected, recovery methods are applied to remove the deadlock.

Recovery from Deadlock

1. Introduction

After detecting a deadlock, the operating system must remove it so blocked processes can continue execution.

This process is called:

Recovery from Deadlock

Recovery techniques break the deadlock cycle and restore normal system operation.

Definition : Recovery from Deadlock is the process of removing deadlocks and restoring processes to normal execution.

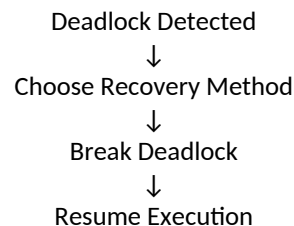
Simple Definition: Recovery from deadlock removes deadlock by terminating processes or reclaiming resources.

Main Idea

After deadlock detection:

- some processes/resources must be sacrificed
- circular waiting must be broken

Recovery Flow



Recovery Methods

There are two major methods.

Method	Idea
Process Termination	Kill processes
Resource Preemption	Take resources back

1. Process Termination

Processes involved in deadlock are terminated.
This releases resources.

Types of Process Termination

Type	Explanation
Abort all deadlocked processes	Kill all involved processes
Abort one process at a time	Gradually remove processes

A. Abort All Processes

All deadlocked processes are terminated immediately.

Example

Suppose:

- P1 and P2 deadlocked

OS:

Terminates P1 and P2

Deadlock removed.

Advantages

Advantage	Explanation
Simple method	Easy implementation
Fast recovery	Immediate solution

Disadvantages

Disadvantage	Explanation
Loss of work	Processes restart
Resource wastage	Partial computation lost

B. Abort One Process at a Time

Processes terminated gradually until deadlock removed.

Example

Terminate P1

Check deadlock

If still exists → terminate P2

Process Selection Criteria

OS selects process based on:

Factor	Meaning
Priority	Important process or not
Execution time	Completed work amount
Resources used	Resource consumption
Remaining execution time	Work left

Advantages

Advantage	Explanation
Less work loss	Fewer processes killed
Better control	Selective recovery

Disadvantages

Disadvantage	Explanation
Slower recovery	Multiple checks needed
Complex decision making	Process selection difficult

2. Resource Preemption

OS forcibly removes resources from processes.

Resources reassigned to break deadlock.

Definition: Resource preemption means temporarily taking resources away from a process.

Example

Suppose:

- P1 holds printer
- P2 waiting

OS:

takes printer from P1

gives to P2

Steps in Resource Preemption

Step	Action
1	Select victim process
2	Preempt resources
3	Rollback process
4	Restart process later

Victim Selection

OS chooses process causing minimum loss.

Rollback

Process state saved and restored later.

Example:

Checkpoint system

Starvation Problem

Same process may repeatedly lose resources.

This causes:

Starvation

Solution to Starvation

Use:

- victim selection limits
- priority aging

Advantages of Resource Preemption

Advantage	Explanation
Less process termination	Processes continue later
Better recovery	Controlled resource handling

Disadvantages of Resource Preemption

Disadvantage	Explanation
Complex implementation	Rollback needed
Data inconsistency risk	Interrupted execution
Starvation possible	Same victim repeatedly selected

Comparison of Recovery Methods

Feature	Process Termination	Resource Preemption
Method	Kill process	Remove resources
Complexity	Simple	Complex
Work loss	High	Lower
Recovery speed	Fast	Moderate

Characteristics of Recovery

Characteristic	Explanation
Reactive approach	Used after detection
Resource release	Breaks circular wait
Process management	Restore execution

Real-Life Analogy

Traffic Jam Example

To clear traffic jam:

- some vehicles redirected
OR
- some vehicles removed

This is similar to recovery methods.

Important Exam Points

- Recovery happens after deadlock detection
- Two methods:
 - Process termination
 - Resource preemption
- Preemption may require rollback
- Starvation possible during recovery
- Abort-all method is fastest

Key Terms

Term	Meaning
Recovery	Remove deadlock
Preemption	Force resource removal
Rollback	Restore previous state
Victim Process	Selected process for recovery

Short Conclusion

Recovery from Deadlock is the process of removing deadlocks after detection. Operating systems use methods such as process termination and resource preemption to break circular waiting among processes. Although recovery restores system operation, it may cause work loss, rollback overhead, and possible starvation.